

Machine Learning Technologies

What are Neural Networks

Donny Hurley

Atlantic Technological University

What are Neural Networks for?

The machine learning algorithms we have seen so far are extremely useful and should be your **first port of call** for any problem.

However, when the number of features is large and the problems become complex and highly non-linear, previous algorithms can struggle.

In these cases a Neural Network can be useful.

Important

While deep neural networks seem to be the only game in town, this is a mistake. Always use a simpler algorithm if it will adequately solve the problem. Simpler models use fewer resources and usually have greater causal reasoning.



- Neural networks arise from attempts to model human/animal brains
- Many models, many claims of biological plausibility
- We will focus on **multi-layer perceptrons** (MLPs)
- We care about their mathematical properties, not their biological plausibility

Why Neural Networks?

There are problems that are difficult for humans but easy for computers

E.g. calculating large arithmetic problems

And there are problems easy for humans but difficult for computers

E.g. recognising a picture of a person from the side

Neural Networks attempt to solve problems that would normally be easy for humans but hard for computers

Example: MNIST

The MNIST dataset is made up of 60k training samples and 10k test samples of handwritten digits. Each sample is a greyscale image of 28×28 pixels — that is 784 features — and the problem is highly non-linear.

Imagine trying to find a hyper-plane in 784-dimensional space that separates each digit. If we try to model the non-linearity with polynomial features the number of features grows explosively. This is where neural networks shine.



Figure: Samples of MNIST Data Set
Image By Josef Steppan - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=64810040>

Mathematical Model of a Neuron

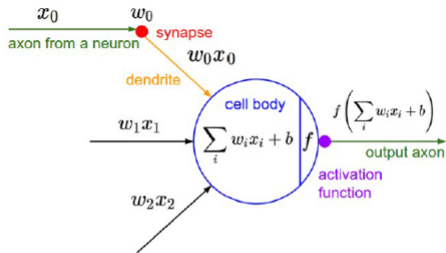
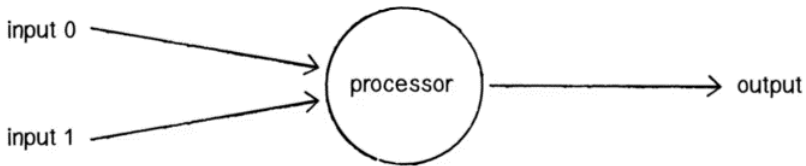


Figure : A mathematical model of the neuron in a neural network

- Neural networks define functions of the inputs (hidden features), computed by neurons
- Artificial neurons are called **units**

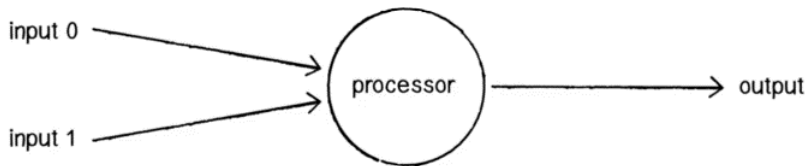
The Perceptron

- We start with the **perceptron**, the fundamental building block
- A perceptron receives one or more inputs, processes them, and produces a single output
- It follows the “feed-forward” model: inputs go in, are processed, a result comes out

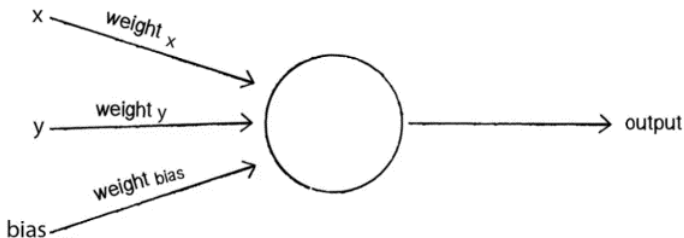


How a Perceptron Works

- 1 **Receive** inputs $x_1, x_2, \dots, x_n$
- 2 **Weight** each input: multiply by $w_1, w_2, \dots, w_n$
- 3 **Sum** the weighted inputs (and add a bias b)
- 4 **Apply** an activation function to produce the output

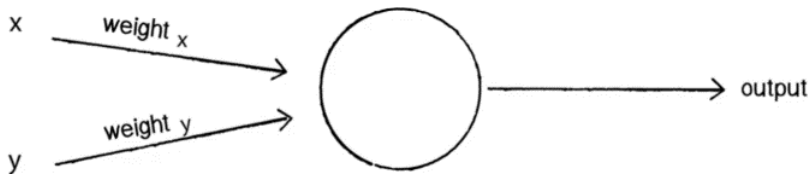


- What if all inputs were zero? The weighted sum would always be zero regardless of the weights — this is a problem.
- To avoid this, we add a **bias** term b
- The bias shifts the activation function and gives the neuron more flexibility



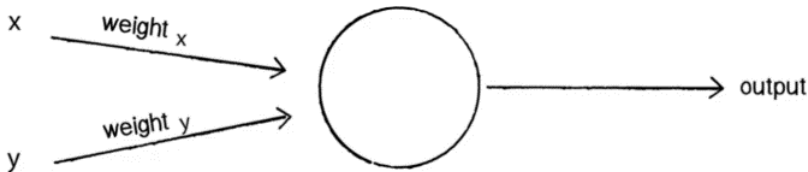
Example: Weighted Inputs

- Say we have two inputs $x = 12$ and $y = 4$
- Each input must first be weighted — multiplied by some value, often between -1 and 1
- Typically we start with random weights
- Say weight_0 is 0.5 and weight_1 is -1
- So $\text{Input}_0 \times \text{Weight}_0$: $12 \times 0.5 = 6$, $\text{Input}_1 \times \text{Weight}_1$: $4 \times -1 = -4$



Example: Activation Function

- The output is generated by passing the weighted sum through an **activation function**
- In the case of a simple binary output the activation function is what tells the perceptron to “fire” or not
- Many activation functions to choose from:
 - Trigonometric, step, logistic/sigmoid, RBF, ReLU



To actually train the perceptron we do the following, starting with random weights:

- ① Provide the perceptron with inputs for which there is a known answer
- ② Ask the perceptron to “guess” the answer
- ③ Compute the error
- ④ Adjust all the weights according to the error
- ⑤ Go to 1 and repeat
 - Repeat until we reach an error we are satisfied with (set beforehand)
 - This is how a single perceptron would work

The Key Insight: A Perceptron *is* Linear Regression + Activation

The computation inside a single perceptron is:

$$\text{output} = \text{activation}\left(\mathbf{w}^T \mathbf{x} + b\right)$$

where $\mathbf{w} = (w_1, w_2, \dots, w_n)$ and $\mathbf{x} = (x_1, x_2, \dots, x_n)$.

You have seen $\mathbf{w}^T \mathbf{x} + b$ before! It is exactly the linear combination used in regression and logistic regression.

Perceptron = Old Friend

- Activation function = **identity** $\Rightarrow$ this is **linear regression**
- Activation function = **sigmoid** $\Rightarrow$ this is **logistic regression**
- Activation function = **step** $\Rightarrow$ classic hard-decision perceptron

A neural network is built by stacking and connecting many of these together.

From Perceptron to Logistic Regression

Recall logistic regression:

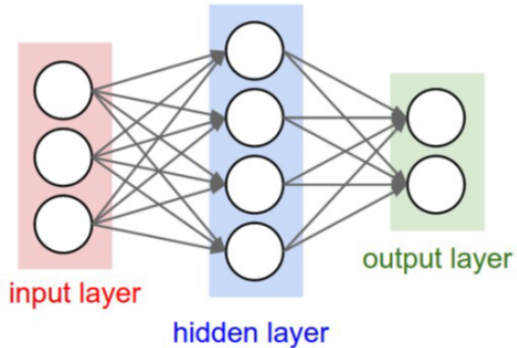
$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

A **single perceptron with a sigmoid activation function** computes exactly this.

So logistic regression is the simplest possible neural network: one input layer, no hidden layers, one output neuron with a sigmoid activation.

When we build deep networks, we are stacking many of these $\mathbf{w}^T \mathbf{x} + b$ operations together, each with their own activation function, allowing the network to learn complex, non-linear representations.

Multi-Layer Perceptron (MLP)



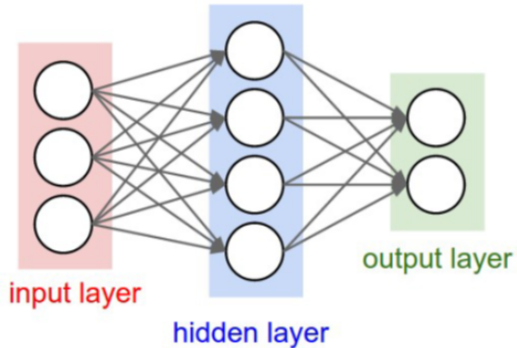
You will have:

- An **input layer** — the features x
- One or more **hidden layers**
- An **output layer**

Hidden layers are called “hidden” because you don’t directly observe their values — only the inputs and outputs.

To create a neural network you link many perceptrons together in layers.

Naming Convention

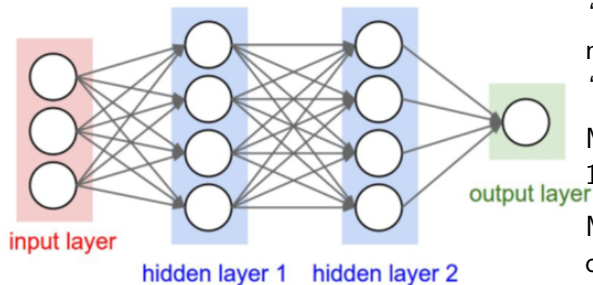


We refer to an N -layer neural network:

- $N - 1$ layers of hidden units
- One output layer
- The input layer does **not** count

This example is a **2-layer** neural network with 3 input units, 4 hidden units and 2 output units.

Going Deeper



A **3-layer** neural network with two hidden layers.

“**Deep Learning**” is simply a neural network with many hidden layers, making it “deep”.

Microsoft’s vision recognition system uses 152 layers!

More layers = more capacity to model complex functions, but also more risk of overfitting.

How Many Weights Does a Fully Connected Network Have?

Weights (one per connection):

$$\underbrace{3 \times 4}_{W_1} + \underbrace{4 \times 2}_{W_2} = 20$$

Biases (one per non-input neuron):

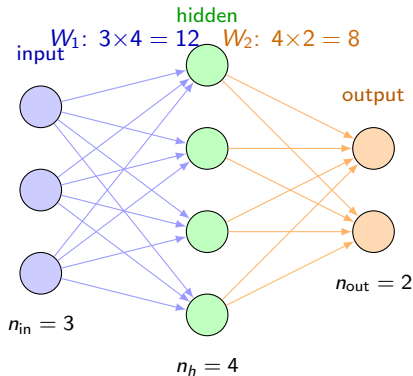
$$4 + 2 = 6$$

Total: 26 parameters

Scale up — 3072 inputs, two hidden layers of 512, 10 outputs:

$$3072 \times 512 + 512 \times 512 + 512 \times 10 = 1,835,520$$

This is why we cannot compute



The Weight Matrices

Each set of connections between two layers is stored as a matrix W .

Input $\rightarrow$ **Hidden** W_1 has shape (hidden $\times$ inputs) = (4×3) :

$$\mathbf{z}_1 = W_1 \mathbf{x} + \mathbf{b}_1 \quad W_1 = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{pmatrix}$$

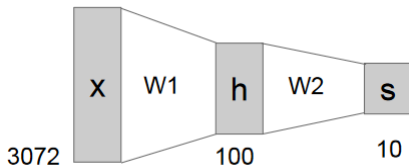
Row i = all weights arriving at hidden neuron i .

Hidden $\rightarrow$ **Output** W_2 has shape (outputs $\times$ hidden) = (2×4) :

$$\hat{\mathbf{y}} = W_2 \text{ReLU}(\mathbf{z}_1) + \mathbf{b}_2$$

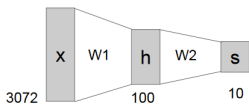
Putting it together: $\hat{\mathbf{y}} = W_2 \text{ReLU}(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$

- (Linear model): $f = W\mathbf{x}$
- (2-layer NN with ReLU): $f = W_2 \max(0, W_1\mathbf{x})$



- (3-layer NN): $f = W_3 \max(0, W_2 \max(0, W_1\mathbf{x}))$

Hidden Layer Intuition



Say x is an image (3072 pixels) to be classified into 10 categories.

Before: went straight from pixels to a category in one matrix multiplication.

Now: hidden vector h (hidden state / hidden layer) of say 100 numbers — a hyperparameter.

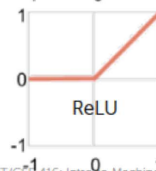
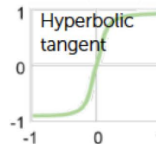
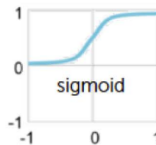
The threshold of 0 (ReLU) deletes negative activations, then a second matrix multiplication does the classification.

The hidden layer could be extracting “interesting” features — e.g. a red car facing forward, or a red car facing right. The elements of h are positive only if they find that “thing” in the image.

Neurons turn on or off if they find what they’re looking for. Then W_2 gives a final score for the classifier.

Which Activation Function?

- **Sigmoid**
 - Historically popular, but (mostly) fallen out of favor
 - Neuron's activation saturates (weights get very large $\rightarrow$ gradients get small)
 - Not zero-centered $\rightarrow$ other issues in the gradient steps
 - When put on the output layer, called "softmax" because interpreted as class probability (soft assignment)
- **Hyperbolic tangent** $g(x) = \tanh(x)$
 - Saturates like sigmoid unit, but zero-centered
- **Rectified linear unit (ReLU)** $g(x) = x^+ = \max(0, x)$
 - Most popular choice these days
 - Fragile during training and neurons can "die off"... be careful about learning rates
 - "Noisy" or "leaky" variants
- **Softplus** $g(x) = \log(1 + \exp(x))$
 - Smooth approximation to rectifier activation



©2018 Emily Fox

STAT/CSE 416: Intro to Machine Learning

Rectified Linear Unit (ReLU)

The ReLU is now the most popular activation function for **hidden layers**.

It is simple but works extremely well in practice.

Alex Krizhevsky showed in the paper that won ImageNet 2012 that ReLU allowed the model to converge **six times faster** than tanh.

Dying ReLU: if a neuron's input is always negative it will never activate (gradient = 0). Leaky ReLU addresses this.

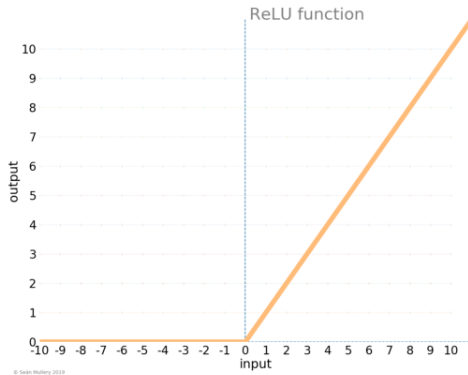


Figure: Rectified Linear Unit

For **hidden layers**:

- **Use ReLU** as your default
- If neurons are “dying”, try **Leaky ReLU**
- **tanh** can work but rarely outperforms ReLU
- **Sigmoid** is generally not recommended for hidden layers (vanishing gradient)

For the **output layer** the choice depends on the problem — we will cover this later.

Interactive Demo: <https://playground.tensorflow.org/>

- Use XOR dataset
- Explore the impact of number of hidden units and activation function

First Look at TensorFlow

We now know enough to build and train a neural network. Full details come later — for Lab 1 here is the recipe:

```
import tensorflow as tf

# 1. Define
model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu',
                           input_shape=(n_features,)),
    tf.keras.layers.Dense(1) # regression: 1 output, no activation
])

# 2. Compile
model.compile(optimizer='adam', loss='mean_squared_error')

# 3. Fit
model.fit(X_train, y_train, epochs=100, verbose=0)

# 4. Predict
y_pred = model.predict(X_test)
```

The Output Layer: it Depends on the Problem

The output layer is where many students get confused. There is no single answer — it depends entirely on what you are trying to predict.

Three cases to understand clearly:

- ① **Regression** — predicting a continuous number
- ② **Binary Classification** — predicting one of two classes
- ③ **Multi-class Classification** — predicting one of $K > 2$ classes

Each case has a different output layer architecture, activation function, and loss function.

Case 1: Regression (1 Output Neuron)

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b \quad \in \mathbb{R}$$

- **Output neurons:** 1
- **Activation: None** (linear / identity) — we want an unbounded real number
- **Loss function:** Mean Squared Error (MSE) or Mean Absolute Error (MAE)

TensorFlow

```
model.add(tf.keras.layers.Dense(1))    # no activation  
model.compile(loss='mean_squared_error', ...)
```

Note: the output can be any real number. If you applied sigmoid here you would constrain the prediction to $(0,1)$ — usually wrong for regression!

Case 2: Binary Classification (1 Output Neuron)

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

- **Output neurons:** 1
- **Activation: Sigmoid** — squashes output to $(0, 1)$, interpreted as $P(y = 1|\mathbf{x})$
- **Loss function:** Binary Cross-Entropy

TensorFlow

```
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))  
model.compile(loss='binary_crossentropy', ...)
```

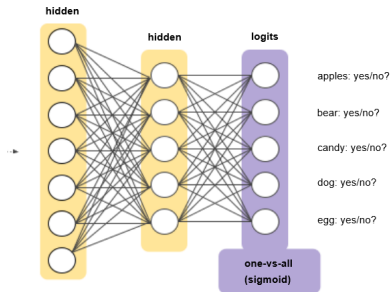
This is logistic regression! The deep network before the output layer is learning a better representation of the inputs, but the final decision is still just a sigmoid on a linear combination.

What about multi-class problems?

- apple, banana, car, walk sign, zebra, zoo
- red, orange, yellow, green, blue, indigo, violet
- animal, vegetable, mineral

We can use One-vs-all as discussed before.

One-vs-all Neural Network



- Create a unique output for each possible class
- Train that on a signal of “my class” vs “all other classes”
- Can do in a deep network, or with separate models

So **one** logistic regression output node in our model for every possible class. They share the internal representation through the rest of the model so these can be trained reasonably efficiently together.

Case 3: Multi-class Classification — The Softmax Function

What if we have $K > 2$ classes (e.g. digits 0–9, or 100 animal species)?

We need K output neurons, one per class, giving us raw scores $z_1, z_2, \dots, z_K$.

We then apply the **Softmax** function to convert these scores into probabilities:

$$\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Key properties:

- Every output is in $(0, 1)$
- All outputs **sum to 1** — they form a proper probability distribution
- The largest z_k gets the largest probability (but all others stay non-zero)

Softmax: A Worked Example

Say we have 3 classes and the network produces raw scores: $z = (2.0, 1.0, 0.1)$

Step 1 — exponentiate each score:

$$e^{2.0} = 7.39, \quad e^{1.0} = 2.72, \quad e^{0.1} = 1.10$$

Step 2 — sum: $7.39 + 2.72 + 1.10 = 11.21$

Step 3 — divide:

$$P(\text{class 1}) = \frac{7.39}{11.21} \approx 0.66 \quad P(\text{class 2}) \approx 0.24 \quad P(\text{class 3}) \approx 0.10$$

They sum to 1.0 and we predict class 1.

Softmax is the generalisation of logistic regression to K classes. For $K = 2$, softmax and sigmoid give identical predictions.

Multi-class Classification in TensorFlow

TensorFlow

```
# K = number of classes
model.add(tf.keras.layers.Dense(K))    # no activation here!
model.compile(
    loss=tf.keras.losses.SparseCategoricalCrossentropy(
                                                from_logits=True),
    ...)
# To get probabilities at prediction time:
probabilities = tf.nn.softmax(model.predict(X_new))
```

Why not put softmax in the model?

From the TensorFlow docs: applying softmax inside the model makes it impossible to provide an exact and numerically stable loss calculation. Always pass `from_logits=True` and apply softmax *after* training when you need probabilities.

What are Logits?

You will see the word **logit** everywhere in TensorFlow. It is worth understanding exactly what it means.

A **logit** is simply the raw, unconstrained output of the final linear layer *before* any activation function is applied:

$$z_k = \mathbf{w}_k^T \mathbf{x} + b_k \in (-\infty, +\infty)$$

Logits (raw scores)

$z = (2.0, 1.0, 0.1)$

Unconstrained, can be any real number

After softmax (probabilities)

$p = (0.66, 0.24, 0.10)$

In $(0, 1)$, sum to 1

Why Does TensorFlow Want Logits?

Why `from_logits=True`?

Computing cross-entropy loss directly from logits is numerically more stable than computing it from probabilities. Probabilities can be very close to 0 or 1, and taking log of a very small number causes floating-point errors. TensorFlow avoids this by working with the raw scores internally.

So: **train with logits** (`from_logits=True`), **apply softmax after** when you need interpretable probabilities.

Common mistake

If you add a softmax activation to the output layer *and* also set `from_logits=True`, TensorFlow will apply the softmax correction twice. Pick one or the other — and prefer `from_logits=True` with no output activation.

Summary: Output Layer Choices

Problem	Output neurons	Activation	Loss
Regression	1	None (linear)	MSE / MAE
Binary classification	1	Sigmoid	Binary cross-entropy
Multi-class (K classes)	K	Softmax [†]	Categorical cross-entropy

[†] Apply softmax after training for numerical stability; use `from_logits=True`.

Note on hidden layers: the activation used in hidden layers (typically ReLU) is completely separate from the output activation. Do not confuse the two.